Task Management API - Stage 4: Full CRUD Implementation (GET, POST, PUT, DELETE)

Overview

This repository houses a lightweight RESTful Task Management API built using Node.js and Express in VS Code on Windows as part of the FlyRank Internship Backend Engineering Track.

Stage 4 completes the full **CRUD (Create, Read, Update, Delete)** lifecycle by adding update (PUT /tasks/:id) and deletion (DELETE /tasks/:id) functionality with strict status code adherence (200 OK, 201 Created, 204 No Content, 400 Bad Request, 404 Not Found).

Technical Stack & Environment

- **Runtime:** Node.js (v24.x)
- **Framework:** Express.js
- **Middleware:** express.json() (Deep Packet Inspection / Body Parser)
- **Data Layer:** In-Memory Volatile Array (RAM Cache)
- **Environment:** VS Code Integrated Terminal (Windows PowerShell pwsh)
- **Verification Tool:** curl.exe (Native Windows Binary)
- **Version Control:** Git

Complete API Catalog & Route Architecture

Method	Endpoint	HTTP Success	HTTP Error	Sysadmin / IT Analogy
GET	/	200 OK	—	IIS Web Site Root Banner / Discovery
GET	/health	200 OK	—	Load Balancer ICMP Heartbeat Ping
GET	/tasks	200 OK	—	Querying service list (Get-Service)

GET	/tasks/:id	200 OK	404 Not Found	Querying Event Viewer ID (Get-WinEvent -InstanceId)
POST	/tasks	201 Created	400 Bad Request	Helpdesk Intake Form Verification & Dynamic Primary Key Allocation
PUT	/tasks/:id	200 OK	400 Bad Request, 404 Not Found	Reconfiguring Active Service Parameters (Set-Service / Registry Key Update)
DELETE	/tasks/:id	204 No Content	404 Not Found	Purging a Service Registration / Deleting ACL Entry (Remove-Item)

Architectural Breakdown: Stage 4 Concepts

1. Reconfiguring Records with PUT /tasks/:id

- **Concept:** PUT updates existing system records in active RAM. The server extracts the :id parameter, verifies existence (404 Not Found), validates the incoming update fields (400 Bad Request), mutates the target record, and returns 200 OK.
- **Sysadmin Analogy:** Executing Set-Service -Name "MyService" -Status Stopped or editing a configuration registry key.

2. Purging State with DELETE /tasks/:id & HTTP 204 No Content

- **Concept:** DELETE locates a target record index in memory using .findIndex() and purges it with .splice().
- **HTTP Status 204 No Content:** Indicates the server processed the deletion request successfully and there is no content/body to return in the response packet.

- **Sysadmin Analogy:** Running a silent removal command (like Remove-Item or rm -f). No text output is required when a target item is deleted.

Stage 4 Verification & Full CRUD Execution Log

1. Launch / Restart Server Daemon

In your **left terminal tab**:

node server.js

2. Step 1: Create a Task (POST /tasks)

In your **right terminal tab**:

curl.exe -i -X POST http://localhost:3000/tasks -H "Content-Type: application/json" -d '{"title":"Buy milk"}'

3. Step 2: Update Task Title (PUT /tasks/4)

curl.exe -i -X PUT http://localhost:3000/tasks/4 -H "Content-Type: application/json" -d '{"title":"Buy fresh organic milk"}'

4. Step 3: Mark Task as Completed (PUT /tasks/4)

curl.exe -i -X PUT http://localhost:3000/tasks/4 -H "Content-Type: application/json" -d '{"done":true}'

5. Step 4: Delete Task (DELETE /tasks/4)

curl.exe -i -X DELETE http://localhost:3000/tasks/4

6. Step 5: State & Error Verification

- **Verify Deletion (GET /tasks):**
  curl.exe -i http://localhost:3000/tasks

- **Verify 404 Error on Deleted Record (GET /tasks/4):**
  curl.exe -i http://localhost:3000/tasks/4

Version Control Checkpoint

git add .
git commit -m "Stage 4: full CRUD"